

Embedded systems I / embedded programming

The inputs / outputs

Pascal Sartoretti

v2.0

I/O : Objectives

Working with inputs and outputs

Organisation of I/O ports
Architecture of I/O pin
Related coding techniques

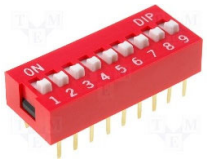


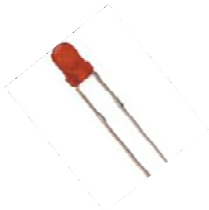
I/O : I/O ports organisation

Why exist I/O ?



Processor always communicates with external word (user interaction, sensors, display …)









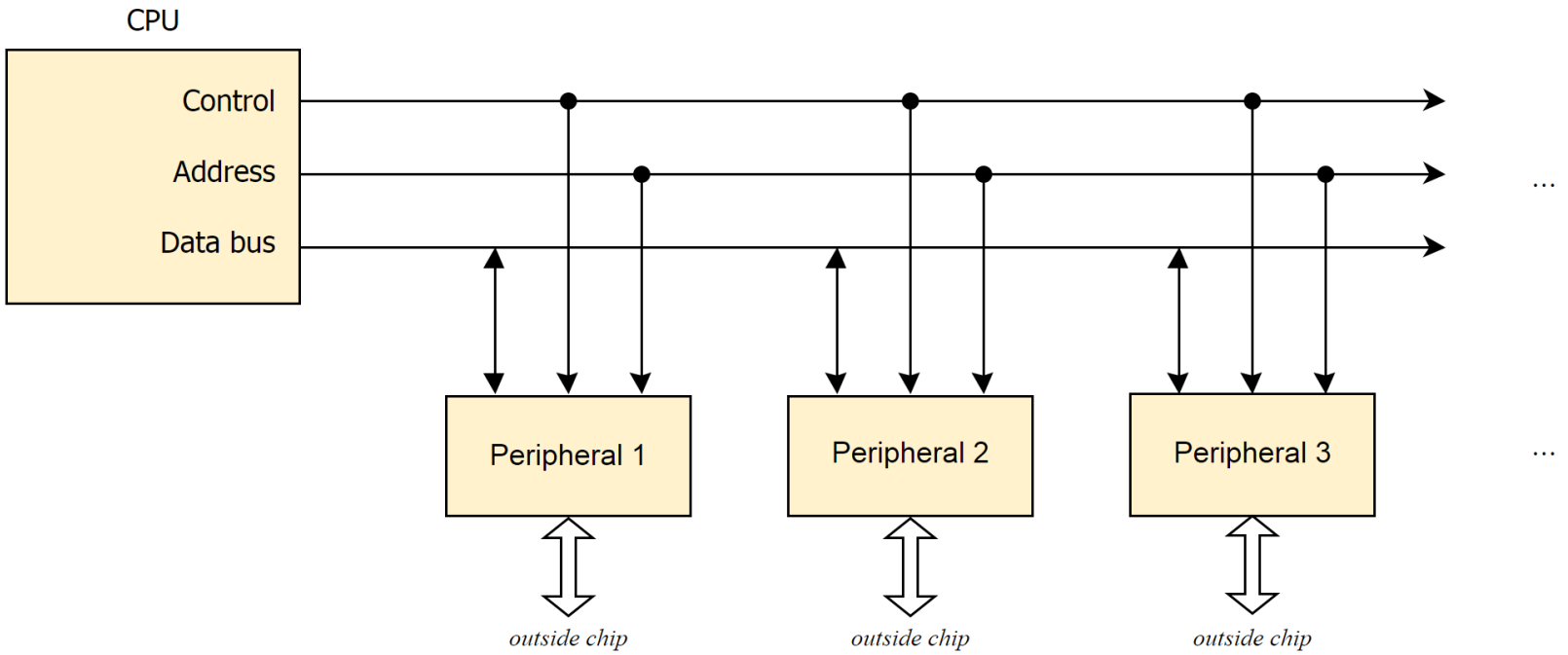






I/O : from CPU to peripherals

General view of peripherals



I/O : STM32U5G9ZJTxQ

10 different ports of 16 bits each (depending on case)

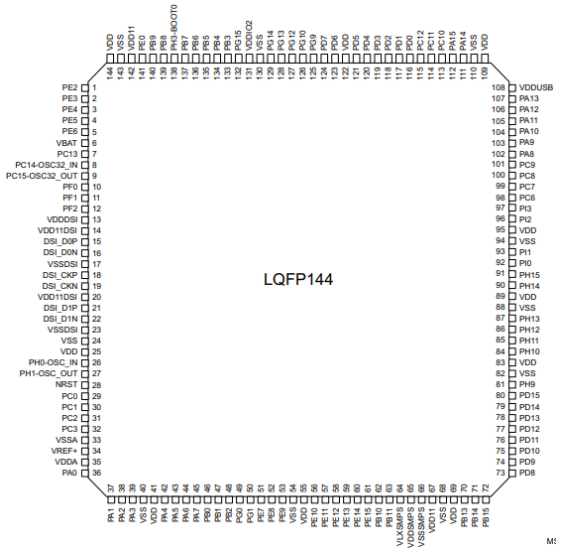
- **PA, PB, PC, … PJ**

Configurable as input or output or special function

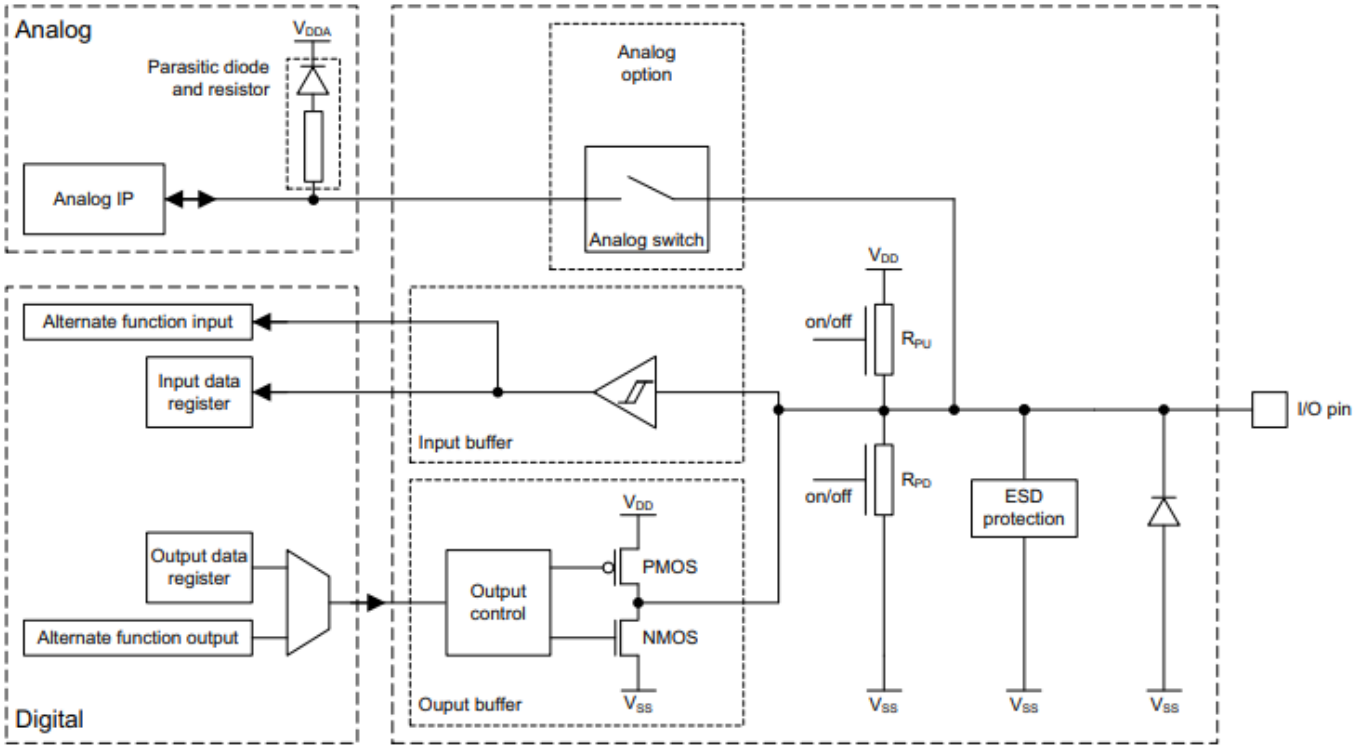
For each port:

- **13 registers (16 or 32 bits)**

GPIOD	0x42020c00
GPIO_MODER @ 0x0	0xffffddf
GPIO_OTYPER @ 0x4	0x00000000
GPIO_OSPEEDR @ 0x8	0x00000000
GPIO_PUPDR @ 0xc	0x00000000
GPIO_IDR @ 0x10	0x00000000
GPIO_ODR @ 0x14	0x00000000
GPIO_BSRR @ 0x18	0x00000000
GPIO_LCKR @ 0x1c	0x00000000
GPIO_AFRL @ 0x20	0x00000000
GPIO_AFRH @ 0x24	0x00000000
GPIO_BRR @ 0x28	0x00000000
GPIO_HSLVR @ 0x2c	0x00000000
GPIO_SECCFGR @ 0x30	0x0000ffff



I/O : STM32 – pin schematic



I/O : STM32 – pin configuration

Each pin of a port can be:

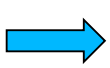
- Input (floating, pull-up, pull-down, analogic, alternate function)

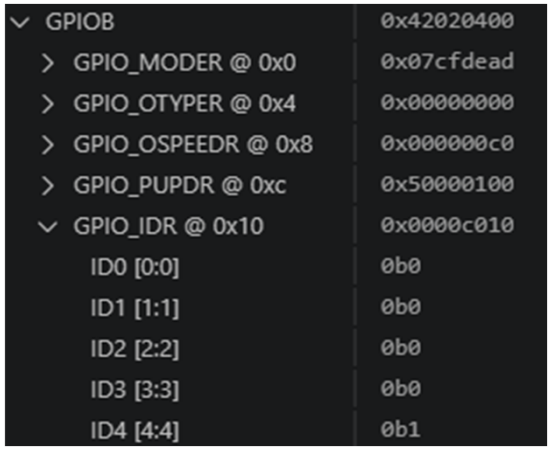
- Output (open-drain, push,-pull, pull-up/down, alternate function)

I/O : STM32 – pin as input

When a pin is configured as digital input, the register we will read is IDR (input data register)

```
GPIOx_IDR (x = A to J)
Address offset = 0x10
Reset value = 0x0000XXXX
```





31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

I/O : STM32 – pin as input

Example of reading port B (@0x42020400 base)

```
void main(void)
{
  uint32_t * ptr = 0x42020400 + 0x10;    // base address + offset of GPIOB_IDR

  while(1)
  {
    uint16_t value = *ptr;                // read this address
  }
}
```

I/O : STM32 – pin as output

When a pin is configured as digital output, two registers can be used:

- **ODR (output data register), like IDR but output**
- **BSRR (bit Set Reset Register)**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w



Atomic operation

I/O : STM32 – pin as input

Example of port B modify (@0x42020400 base)

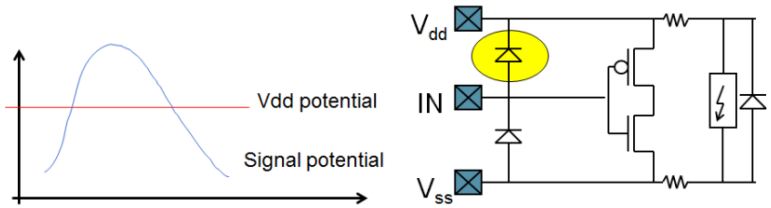
```
void main(void)
{
  uint32_t * ptr = 0x42020400 + 0x18;    // base address + offset of GPIOB_BSRR

  while(1)
  {
    *ptr = 0x00020001;                   // set bit ? and clear bit ?
  }
}
```

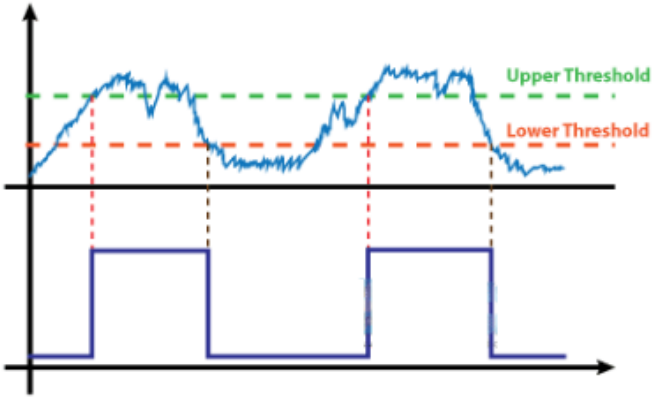
What happens ?

I/O : I/O ports electrical input view

Diode protected input

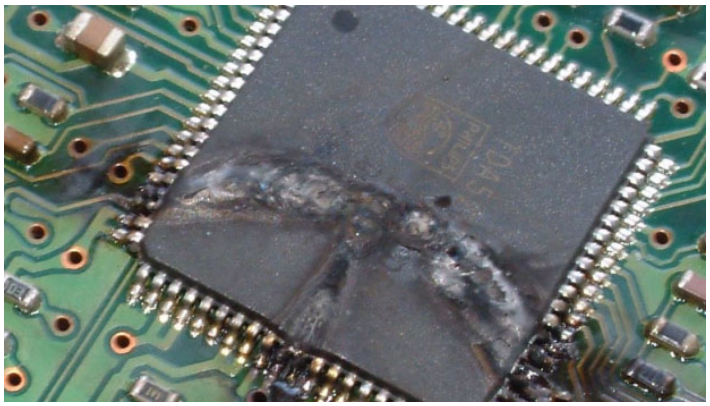


Schmitt trigger input

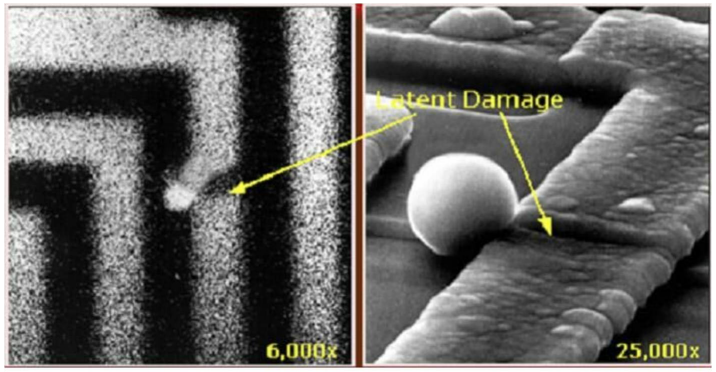


I/O : I/O ports electrical ESD damage

Direct faults : 30%



Latent faults : 70%





I/O : CMOS transistors

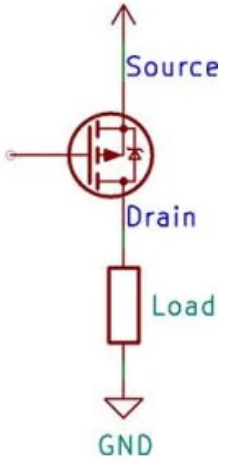
Switch usage in digital

P-channel MOSFET
- **Drives when VGS < 0**

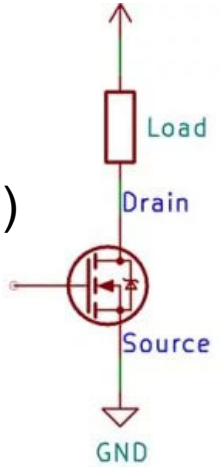
N-channel MOSFET
- **Drives when VGS > 0**

- ***Driving*** *≈ wire between D and S (RDS ON)*
- ***Not driving*** *≈ D and S not connected (RDS OFF)*

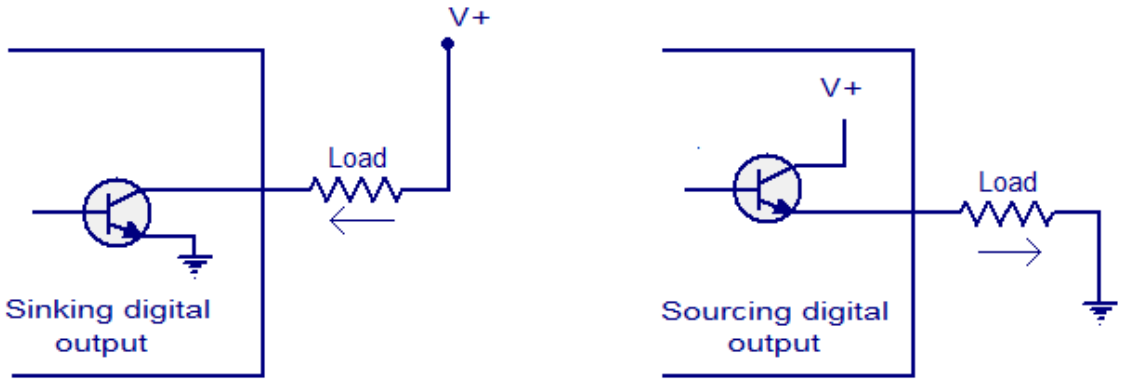
P-Channel symbol
(source always to VCC)
Drives when G = GND



N-Channel symbols
(source always to GND)
Drives when G = VCC

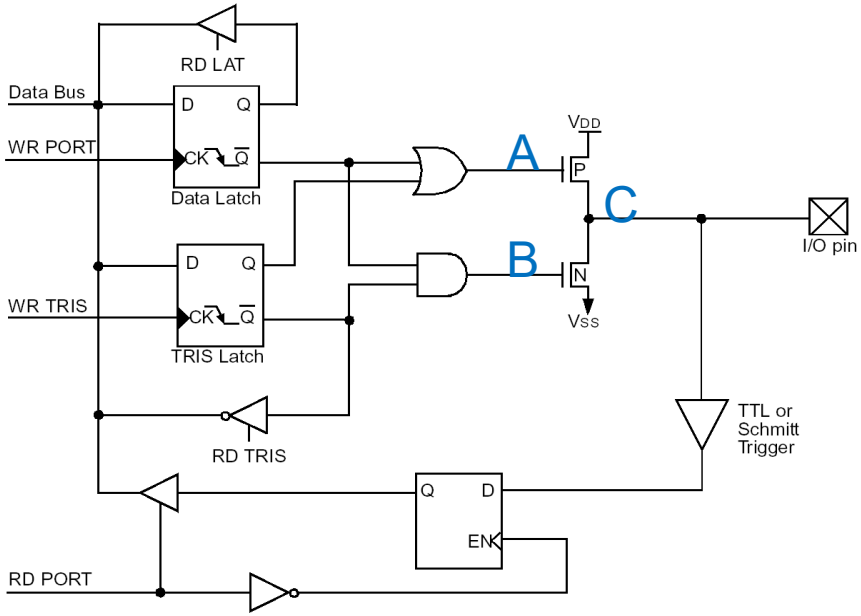


I/O : current sourcing vs current sinking



I/O : exercise

Draw the truth table of A, B and C for the various TRIS and PORT values (note that C has possibles values of 0, 1 and Z)



DATA LATCH (Q)	TRIS LATCH (Q)	A	B	C
0	0			
0	1			
1	0			
1	1			

- **P-Channel symbol**
  - Source always to VCC (Vdd)
  - Drives when G = GND (0)
- **N-Channel symbol**
  - Source always to GND (Vss)
  - Drives when G = VCC (1)

I/O : more electrical information's

Sink and source not always the same

Symbol	Parameter	Conditions	Min	Max	Unit
V_{OL}	Output low-level voltage	CMOS port(4), $\|I_{IO}\|$ = 8 mA, 2.7 V ≤ V_{DDIOx} ≤ 3.6 V	-	0.4	V
V_{OH}	Output high-level voltage		V_{DDIOx} - 0.4	-	
V_{OL}(5)	Output low-level voltage	TTL port(4),$\|I_{IO}\|$ = 8 mA, 2.7 V ≤ V_{DDIOx} ≤ 3.6 V	-	0.4	
V_{OH}(5)	Output high-level voltage		2.4	-	
V_{OL}(5)	Output low-level voltage	All I/Os, $\|I_{IO}\|$ = 20 mA, 2.7 V ≤ V_{DDIOx} ≤ 3.6 V	-	1.3	
V_{OH}(5)	Output high-level voltage		V_{DDIOx} - 1.3	-	
V_{OL}(5)	Output low-level voltage	$\|I_{IO}\|$ = 4 mA, 1.58 V ≤ V_{DDIOx} ≤ 3.6 V	-	0.4	
V_{OH}(5)	Output high-level voltage		V_{DDIOx} - 0.4	-	
V_{OL}(5)	Output low-level voltage	$\|I_{IO}\|$ = 1 mA, 1.08 V ≤ V_{DDIOx} <3.6 V	-	0.4	
V_{OH}(5)	Output high-level voltage		V_{DDIOx} - 0.4	-	

I/O : STM32 HAL GPIO functions

To access to the GPIO, STM offers Hardware Abstraction Layer functions.

- `void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);`
- `GPIO_PinState HAL_GPIO_ReadPin(const GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);`
- `void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);`
- `...`

To access to a port, the supply of this port must be activated with:

- `__HAL_RCC_GPIOE_CLK_ENABLE();  // GPIOE for example`

I/O : STM32 HAL GPIO functions

Definitions:

- `GPIO_TypeDef (GPIOA, GPIOB, ...)`
- `GPIO_Pin (is the pin position, (1 << 0) to (1 << 15), or GPIO_PINx)`
- `GPIO_PinState`

```
typedef enum
{
    GPIO_PIN_RESET = 0U,
    GPIO_PIN_SET
} GPIO_PinState;
```

I/O : STM32 HAL GPIO functions

To configure a GPIO on the STM32.

- `void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, const GPIO_InitTypeDef *pGPIO_Init);`

Activate RCC clock before !

```
typedef struct
{
 uint32_t Pin;        // GPIO pins -> @ref GPIO_pins
 uint32_t Mode;       // Operating mode -> @ref GPIO_mode
 uint32_t Pull;       // Pull/up/down activation -> @ref GPIO_pull
 uint32_t Speed;      // Speed -> @ref GPIO_speed
 uint32_t Alternate; // see @ref GPIOEx_Alternate_function_selection
} GPIO_InitTypeDef;
```

```
GPIO_Typedef
GPIOA
GPIOB
...
```

I/O : STM32 HAL GPIO functions

GPIO_InitTypedef parameters

```
GPIO_pins (multiple)
GPIO_PIN_0
GPIO_PIN_1
...
```

```
GPIO_pull (25 kOhms to 60 kOhms)
GPIO_NOPULL
GPIO_PULLUP
GPIO_PULLDOWN
```

```
GPIO_mode
GPIO_MODE_INPUT
GPIO_MODE_OUTPUT_PP
GPIO_MODE_OUTPUT_OD
GPIO_MODE_AF_PP
GPIO_MODE_AF_OD
GPIO_MODE_ANALOG
GPIO_MODE_IT_RISING
GPIO_MODE_IT_FALLING
...
```

```
GPIO_speed (caution with pin capacitance)
GPIO_SPEED_FREQ_LOW      // up to 10 MHz
GPIO_SPEED_FREQ_MEDIUM   // up to 50 MHz
GPIO_SPEED_FREQ_HIGH     // up to 100 MHz
GPIO_SPEED_FREQ_VERYHIGH // up to 160 MHz
```

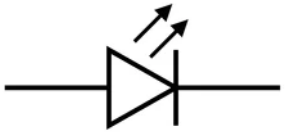
```
GPIO_alternate (depend on selected pin)
...
A lot of options
...
```

I/O : Exercise

Draw a schematic to connect a push button to a STM32 GPIO powered with 3.3V



Modify the schematic to connect a LED to another STM32 GPIO



LED : 1.7V @ 4mA

I/O : exercise

Write the code reflect the state of the button on the LED (pressed = LED on)

```
void main(void)
{
 GPIO_InitTypeDef GPIO_InitStruct = {0};
 __HAL_RCC_GPIO  _CLK_ENABLE();
 GPIO_InitStruct.Pin =                    ;
 GPIO_InitStruct.Mode =                   ;
 GPIO_InitStruct.Pull =                   ;
 GPIO_InitStruct.Speed =                  ;
 HAL_GPIO_Init(PORT  , &GPIO_InitStruct);

 __HAL_RCC_GPIO  _CLK_ENABLE();
 GPIO_InitStruct.Pin =                    ;
 GPIO_InitStruct.Mode =                   ;
 GPIO_InitStruct.Pull =                   ;
 GPIO_InitStruct.Speed =                  ;
 HAL_GPIO_Init(PORT  , &GPIO_InitStruct);
 ...
```

I/O : exercise

```
...
   while(1)
   {









   }
}
```

I/O : setting a bit on a port without BSRR

```
GPIOD->ODR = 0x0010;                // set bit 4 on PORTD and clear others

GPIOD->ODR = GPIOD->ODR | 0x0010; // set bit 4 on PORTD only

GPIOD->ODR = GPIOD->ODR | (1<<4);  // set bit 4 on PORTD more nice
```

What is really done :

```
ODR    : xxxx xxxx
Mask   : 0001 0000
================== (or operation)
ODR    : xxx1 xxxx
```

I/O : clearing a bit on a port without BSRR

```
GPIOD->ODR = 0xBF;                  // clear bit 6 on PORTD and set others

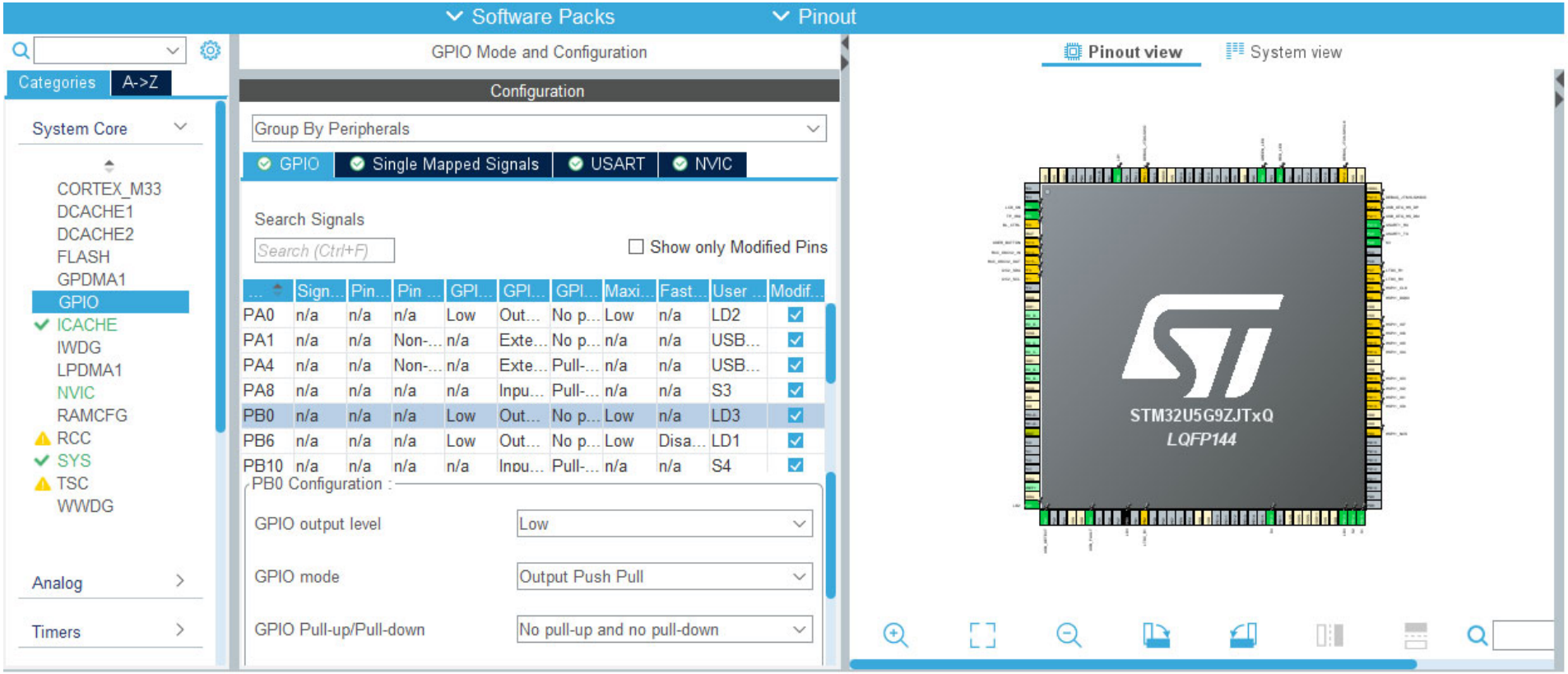
GPIOD->ODR = GPIOD->ODR & ~0x40;    // clear bit 6 on PORTD only

GPIOD->ODR = GPIOD->ODR & ~(1<<6);// clear bit 6 on PORTD more nice
```

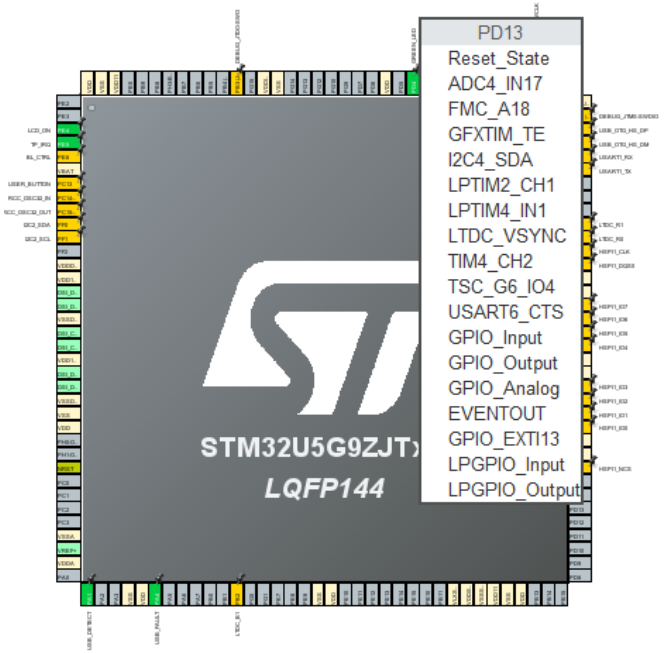
What is really done :

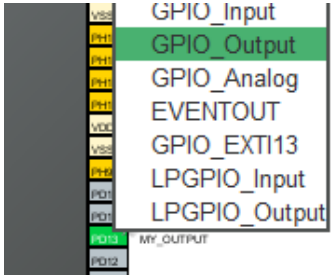
```
ODR    : xxxx xxxx
Mask   : 0100 0000
Invert: 1011 1111
================== (and operation)
ODR    : x0xx xxxx
```

I/O : Use device configuration tool



I/O : Use device configuration tool





PD13 Configuration :

GPIO output level	Low
GPIO mode	Output Push Pull
GPIO Pull-up/Pull-down	No pull-up and no pull-down
Maximum output speed	Low
User Label	MY_OUTPUT